Week 8: Solving Bigger Problems

Ellen Bledsoe

Solving Bigger Problems

This week, we are going to take a break learning commands to talk about general approaches to programming.

Real computational tasks tend to be complicated. To accomplish them, you need to *think before you code*.

Set Up

For this lesson and the assignment, we will be using a number of datasets, including the 3 datasets with Portal data that we are already somewhat familiar with.

Let's read in the `tidyverse` and download our datasets to get started.

```
# packages
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
# data
download.file("https://ndownloader.figshare.com/files/2292172",
  "data/surveys.csv")
download.file("https://ndownloader.figshare.com/files/3299474",
  "data/plots.csv")
download.file("https://ndownloader.figshare.com/files/3299483",
  "data/species.csv")
download.file("https://datacarpentry.org/semester-biology/data/mammal-size-data-clean.tsv",
  "data/mammal-size-data-clean.txt")
```

Problem Decomposition Steps

One great way to work through the steps of coding is to break down the problem into smaller, more manageable chunks. This can also help you get an idea of the order in which to approach the coding.

Here are some good guidelines:

1. Understand the problem
    a. Restate the problem in your own words
    b. Determine the inputs and outputs
    c. Ask for clarification
2. Break the problem down
    a. Write down the pieces, on paper or as comments in a file
    b. Break complicated pieces down until all pieces are small and manageable
3. Code one small piece at a time
    a. Test it on its own
    b. Fix any problems before proceeding

Let's Practice!

Take a look at Question 4 in the assignment. Spend a few minutes reading through the question, restate the question in your own words, and then break down the steps.

Iteration

As you've probably discovered by now, programming requires a lot of iteration to get code to do exactly what you want it to do and produce the output you are looking for.

When writing more and more complex code, it can help to start simple and build the code up.

Write a simple version -> Make sure it works -> Make sure you understand it -> Modify it to make it more complicated -> Repeat until finished

Perhaps we want to calculate the average hindfoot length in cm for each rodent species at Portal.

Let's start by breaking down the problem. Then, we can add one piece of code at a time.

```
# join species and surveys data
# filter for rodent species only
# convert hindfoot length to cm
# group data by species
# summary data of hindfoot length

surveys <- read_csv("data/surveys.csv")
```

```
Rows: 35549 Columns: 9
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (2): species_id, sex
dbl (7): record_id, month, day, year, plot_id, hindfoot_length, weight

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
species <- read_csv("data/species.csv")
```

```
Rows: 54 Columns: 4
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (4): species_id, genus, species, taxa

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
avg_rodent_hf_cm <- surveys |>
  inner_join(species, join_by(species_id)) |>
  filter(taxa == "Rodent") |>
  mutate(hindfoot_cm = hindfoot_length / 10) |>
  group_by(species_id) |>
  summarise(mean_hf = mean(hindfoot_cm, na.rm = TRUE))
```

Along the way, it's a good idea to check that each step did what you expected. For example, did our join keep all of the rows in `surveys`? Let's check the number of rows before and after the join.

```r
# rows in the original data
nrow(surveys)
```

```
[1] 35549
```

```r
# rows after the join
nrow(inner_join(surveys, species, join_by(species_id)))
```

```
[1] 34786
```

We lost 763 rows! The `inner_join()` only keeps rows that have a match in both tables, and 763 records in `surveys` have no `species_id`, so they can't match anything in `species`. That is fine for what we are doing here, but it is good to know it happened rather than discover it later.

Debugging

A major part of the iterative process of coding is fixing errors that come up! You've already had quite a bit of practice with this working on your assignments, but let's talk about some general strategies together.

1. Be a scientist. Develop hypotheses!

- Hypothesize about what is wrong.
- Make one change that is expected to fix error.
- Check if change worked/fixed error.

2. Do not change something without a reason.

- See where the code failed.

- Read the error message.
- Observe what the code is doing.
- Look at the current state of the environment. This gives you a current snapshot of what is going on.
- Talk through the code
- Rubber duck programming

3. Run the code line by line, checking each step individually.
4. Use generative AI (e.g., ChatGPT)–with caution. We'll talk about how to do this well in Using Generative AI Effectively below.

Example

Below, we have a code chunk with some code that is meant to summarize some of the Portal data and then plot it. However, as you'll see when you try to run the code, we have some errors.

First, copy the code into a new code chunk below. Spend a few minutes working through this code chunk to debug it. Utilize some of the strategies above.

```
# Code chunk to debug: leave as is for reference.
surveys <- read_csv('surveys.csv')
species <- read_csv('species.csv')

do_counts_by_year <- survey |>
  filter(species = "DO") |>
  group_by(year)
  count()

ggplot(do_count_by_year, aes(x = year, y = count) |>
  geom_point() +
  geom_line()
  lab(x = "Year", y = "Count") +
  theme_bw()
```

Insert a code chunk here. Copy the code from above and work to debug it.

```
# point to correct directory
surveys <- read_csv('data/surveys.csv')
```

```
Rows: 35549 Columns: 9
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (2): species_id, sex
dbl (7): record_id, month, day, year, plot_id, hindfoot_length, weight

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
species <- read_csv('data/species.csv')
```

```
Rows: 54 Columns: 4
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (4): species_id, genus, species, taxa

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# don't save as new object until the pipes run and produce expected result
do_counts_by_year <- surveys |> # correctly reference `surveys`
  filter(species_id == "DO") |> # correctly reference `species_id` and ==
  group_by(year) |>             # add a pipe
  count()

# correctly reference object name and correct column name for y
ggplot(do_counts_by_year, aes(x = year, y = n)) +
  geom_point() +
  geom_line() +                   # add missing +
  labs(x = "Year", y = "Count") + # correct function is labs
  theme_bw()
```



If you want to, you can also copy the code into your Gen AI of choice and see what it says. We'll come back to this after we talk about how to write a better prompt.

Finding Help

In this class, I've provided the instructor versions of the lessons in class for you to use as reference sheets. They are super helpful, but they won't always have exactly what you need, especially when you are working with your own datasets.

The good news is that there is help! In fact, professional programmers use Google regularly; I am constantly googling how to achieve certain tasks, including for the lessons and assignments in this class!

There is a bit of an art to learning how to search, read, and apply online help. Let's talk through a few tips that you might find helpful.

In the Search:

- Get the vocabulary right
- Avoid extra words
- Specify the programming language (and specify `tidyverse` if that matters to you)
- These suggestions are also effective with generative AI.

Results:

Help sites

- Read the question
- Look at the setup
- Check the age (when was it answered?)
- Check the top 2-3 answers
- Glance at the comments
- Test & modify the example
  - Don't blindly paste things you don't understand
  - Reputation can help (e.g., StackOverflow answer with lots of positive votes)

Blog posts

- Read the question
- Look at the setup
- Use `Find` (`Ctrl` + `F` or `Cmd` + `F`)

Documentation

- Use `Find` (`Ctrl` + `F` or `Cmd` + `F`)
- Focus on the examples

Testing and Modifying Answers

Often, we won't be able to tell right away if the code provided in the answer will work for our specific data or situation. We always need to test it out.

First, if given a reproducible example (reprex), meaning it has data and code you can copy and paste to run, do so! This can help with understanding what the code is doing and whether or not it will work in your case.

From there, you can begin to modify the example to include your data and specific values.

Using Generative AI Effectively

Generative AI tools (e.g., ChatGPT, Claude, Gemini, Copilot) can be a really useful coding partner. I use them regularly to draft code, decode error messages, and remember how to do things I've done before. But they sound confident even when they are wrong, and they know nothing about your data beyond what you tell them. You are responsible for the code–and the science.

Think Before You Prompt

- Gen AI works best on small, well-defined tasks. "Analyze my data" will get you generic code; "calculate the mean weight of each species" will get you something useful.
- Break the problem down and try it yourself first, like we practiced earlier. Then hand over one small piece at a time.
- Ask for code, not answers. If you ask an AI to analyze your data directly, it may give you numbers that look right but aren't. Ask it to write code, then run that code yourself.

Give the AI Context

An AI assistant can't see your computer, your files, or your data unless you show it. A good prompt usually includes:

1. The language and packages you are using
2. The structure of your data, plus anything only you know (like units)
3. Your goal and what the result should look like
4. What you have tried, including the *exact* error message
5. Any constraints (e.g., "use only `dplyr` functions")

Compare these two prompts for the broken code from our debugging example. First, a weak one:

> Fix this code. [code]

And now a better one:

> I'm using R and the tidyverse. `surveys` is a data frame with one row per animal captured and columns including `year` and `species_id`. I want to count the records of species `DO` in each year and plot the counts over time as points connected by a line. The code below gives errors. Please explain what's wrong and show me the corrected code. [code]

You don't have to share all of your data. `glimpse()` and `head()` show its structure, and pasting that output is usually enough.

```
# column names and types
glimpse(surveys)
```

```
Rows: 35,549
Columns: 9
$ record_id       <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,~
$ month           <dbl> 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, ~
```

```
$ day             <dbl> 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16~
$ year            <dbl> 1977, 1977, 1977, 1977, 1977, 1977, 1977, 1977, 1977, ~
$ plot_id         <dbl> 2, 3, 2, 7, 3, 1, 2, 1, 1, 6, 5, 7, 3, 8, 6, 4, 3, 2, ~
$ species_id      <chr> "NL", "NL", "DM", "DM", "DM", "PF", "PE", "DM", "DM", ~
$ sex             <chr> "M", "M", "F", "M", "M", "M", "F", "M", "F", "F", "F",~
$ hindfoot_length <dbl> 32, 33, 37, 36, 35, 14, NA, 37, 34, 20, 53, 38, 35, NA~
$ weight          <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA~
```

```
# a few example rows
head(surveys, 5)
```

```
# A tibble: 5 x 9
  record_id month   day  year plot_id species_id sex   hindfoot_length weight
      <dbl> <dbl> <dbl> <dbl>   <dbl> <chr>      <chr>           <dbl>  <dbl>
1         1     7    16  1977       2 NL         M                  32     NA
2         2     7    16  1977       3 NL         M                  33     NA
3         3     7    16  1977       2 DM         F                  37     NA
4         4     7    16  1977       7 DM         M                  36     NA
5         5     7    16  1977       3 DM         M                  35     NA
```

Treat it like a conversation: follow up with new error messages, change one thing at a time, and start a new chat if you are going in circles.

Use It Responsibly

- Anything you paste into a Gen AI tool is sent to a company's server and may be stored or used for training. Don't paste unpublished, sensitive, or restricted data (e.g., human subjects data, exact locations of threatened species) or anything with passwords or API keys.
- The chat isn't a reproducible record; your script is. Save the code you have checked, and note where AI helped:

  ```
  # Drafted with help from Gen AI (tool name, date)
  # Checked: row counts before/after join; compared one mean by hand
  ```

- Follow the rules in the syllabus. Many journals and funders also require you to disclose your use of Gen AI.

Verify Everything

Gen AI tools generate responses that are likely to *sound* right, which isn't the same as *being* right. You might get functions that don't exist, outdated code, or code that runs without errors but doesn't do what you asked. That last one is the most dangerous with data, because there is no error message to warn you.

Treat AI-generated code like your own: *Make sure it works -> Make sure you understand it.*

1. Read it and make sure you can explain every line (look up what you don't recognize with `?`)
2. Run it in pieces, one step at a time
3. Check the output against what you expect, like we did with `nrow()` earlier (`NA`s? extreme values? does the plot look right?)
4. Check it against your data: the AI doesn't know your units or how missing values are coded

Suppose you asked an AI assistant to "calculate the mean weight of each species in the Portal data," and it gave you this code:

```
avg_weight <- surveys |>
  inner_join(species, join_by(species_id)) |>
  group_by(species_id) |>
  summarise(mean_weight = mean(weight, na.rm = TRUE))
```

It runs without errors and returns numbers. But can we trust them? What would you want to know before using these results?

```
# do the results look reasonable?
avg_weight |>
  filter(is.nan(mean_weight))
```

```
# A tibble: 23 x 2
   species_id mean_weight
   <chr>            <dbl>
 1 AB                 NaN
 2 AH                 NaN
 3 AS                 NaN
 4 CB                 NaN
 5 CM                 NaN
 6 CQ                 NaN
 7 CS                 NaN
 8 CT                 NaN
```

```
 9 CU                  NaN
10 CV                  NaN
# i 13 more rows
```

Many species (mostly birds and reptiles) have no weight measurements at all, so their "mean" is `NaN`. Setting `na.rm = TRUE` hid the `NA`s, but it didn't make the missing data go away, and the AI didn't flag it.

Use AI to Learn, Not Just to Get Answers

Gen AI can be a patient tutor if you ask it to be. Try prompts like:

- "Explain this code line by line. I'm new to R."
- "Don't give me the solution yet. Give me a hint about what to try next."
- "What is the difference between `inner_join()` and `left_join()`? Show me a small example."

A good test: could you write this code again without the AI?

Let's Practice!

Go back to the broken code in the Debugging example and ask a Gen AI tool for help, this time with a better prompt using the tips above. If you already tried a quick prompt earlier, compare the two responses: which was more useful, and which explained *why* the code was broken? Then run the code and make sure you can explain every change the AI made.